

Sistemas Operativos

2º Parcial 1C2026 TT – Resolución

Aclaración: La mayoría de las preguntas o ejercicios no tienen una única solución. Por lo tanto, si una solución particular no es similar a la expuesta aquí, no significa necesariamente que la misma sea incorrecta. Ante cualquier duda, consultar con el/la docente del curso.

Teoría

1-

- a) Fragmentación externa
- b) Segmentacion
- c) Thrashing (sobrepaginación).
- d) Tabla de páginas

2-

- a) Contigua
- b) Soft Links
- c) Soft Links
- d) Tabla local de archivos abiertos por proceso

3-

	1 nivel	Multinivel	Invertida
Consumo memoria	Alto, dado que la tabla es tan grande como la cantidad de páginas del proceso.	Bajo / Intermedio, dado que si bien las tablas son tan grandes como las páginas necesarias, una parte se encuentra en disco.	Bajo / Intermedio, dado que la tabla tiene un tamaño fijo, independientemente del tamaño de los procesos.
Accesos para traducción	Minima (1 acceso para traducción)	Intermedia, dependiendo de la cantidad de niveles	Intermedia / baja, dependiendo de si la pagina es encontrada en el primer acceso o si es necesario recorrer el overflow de la tabla.

4- La serie de pasos que se realizan en el escenario propuesto son:

1. La MMU lee la tabla de páginas y detecta que la página no está presente en RAM (bit de presencia en 0)
2. Se genera una interrupción indicando Page Fault
3. El SO inicia la lectura de la página desde el almacenamiento secundario
4. El proceso pasa a estar bloqueado
5. El DMA realiza transferencia de disco a memoria
6. Se produce una interrupción, la cual es atendida por el SO
7. Se actualiza la tabla de páginas
8. El proceso vuelve a Ready
9. La instrucción se ejecuta.

Si tenemos en cuenta la existencia de una caché tipo TLB, previo al paso 1, se buscaría la página a leer en la misma, donde no la encontrará en este escenario, luego junto al paso 7 se actualiza la TLB al igual que la tabla de páginas.

5-

a)

La estrategia consiste en utilizar el sistema de paginación con memoria virtual para que los procesos puedan operar sobre archivos, trabajando con syscalls que permiten asumir que dicho contenido se encuentra cargado en su totalidad en memoria principal. El SO divide el archivo mapeado en páginas, e irá cargando a memoria aquellas que sean solicitadas por el proceso, de forma idéntica a la que se usaría si fueran páginas de datos, código o alguna otra parte tradicional del proceso.

Ejemplo:

```
void * p = mmap("a.txt") // Retorna un puntero al primer byte del
archivo
P[1000] = 543 // Se escribe el valor 543 en la posición 1000 del
archivo
```

b)

Los locks permiten evitar las condiciones de carrera en la manipulación de archivos, permitiendo lograr la mutua exclusión.

Al estar dedicados exclusivamente para proteger archivos, ofrecen particularidades que ayudan a la eficiencia y a garantizar la integridad del recurso:

- El uso de locks obligatorios le permite a un proceso operar de forma segura independientemente del uso indebido de otros procesos
- El uso de los compartidos permite un uso eficiente en términos de concurrencia, permitiendo que varios procesos puedan acceder a un archivo en modo solo lectura.

Práctica

1-

a)

Para llegar al archivo C:\System32\Win\lib.dll, hay que pasar por 3 directorios (C, System32 y Win).

Es necesario leer el contenido de cada uno de ellos, los cuales ocupan un bloque de datos cada uno. Por lo tanto:

- Lectura del directorio C (1 cluster) (recordar que dicho nro de cluster ya es conocido)
- Lectura del directorio System32 (1 cluster)
- Lectura del directorio Win (1 cluster)

Con la lectura del directorio Win, conoceremos el nro del primer cluster del archivo lib.dll, sin embargo necesitamos leer el 2do cluster (dado que el byte 5000/4096 = 1,2...). Por lo tanto, vamos a la posición de la tabla FAT correspondiente al 1er cluster, para descubrir el cluster siguiente. Recién entonces podemos leer el cluster 2do del archivo (1 cluster leído).

En total, 4 clusters leídos.

b) Del analisis anterior se desprende que fue necesario **un solo acceso** a la tabla fat.

c) Tamaño maximo teórico del fs: $2^{28} \times 2^{12} = 1 \text{ TiB}$

d) El archivo mas grande posible en este volumen (equivalente al tamaño maximo real de un archivo) es el minimo entre el maximo teorico (1 TB) y el volumen (2 TB) => **1 TiB**

2-

a)

Se deja un grafico que respeta el enunciado especificado. Los nros de bloques e inodos estan elegidos al azar, pero respetando la coherencia de las relaciones entre ellos de acuerdo al enunciado.

Inodo 1 ("/")	bloque 10		Inodo 2 ("etc")	bloque 11	
atributos..	<i>nombre</i>	<i>Inodo</i>	atributos..	<i>nombre</i>	<i>Inodo</i>
puntero: bloque 10	etc	2	puntero: bloque 11	fixture.txt	5
	var	3			
	temp	4	Inodo 3 ("var")	bloque 12	
			atributos..	<i>nombre</i>	<i>Inodo</i>
Inodo 5 ("fixture.txt")			puntero: bloque 12	h-fixture.txt	5
atributos...					
hard links: 2			Inodo 4 ("temp")	bloque 13	
puntero: bloque 20			atributos..	<i>nombre</i>	<i>Inodo</i>
			puntero: bloque 13	s-fixture.txt	6
Inodo 6 ("s-fixture.txt")	bloque 14				
atributos...	"/etc/fixture.txt"				
tipo: soft-link					
hard links: 1					
puntero: bloque 14					

b) El soft link "/temp/s-fixture.txt" permite distinguir entre el original y el link, debido a que es un inodo separado cuyo tipo de archivo así lo especifica.

c) Eliminar /etc/fixture.txt implicaría que el soft link quede inválido (aunque seguiría existiendo como archivo). El hard link /var/h-fixture.txt seguiría permitiendo acceder al archivo original.

d) Deberían eliminarse /etc/fixture.txt y /var/h-fixture.txt

3-

Para cada referencia, es necesario obtener el número de página correspondiente. Para ello, primero necesitamos conocer el tamaño de la página.

Sabemos que con direcciones lógicas de 20 bits, hay un máximo de 128 páginas, o sea un máximo de 2^7 páginas. Por lo tanto, $20 \text{ bits} - 7 \text{ bits} = 13 \text{ bits}$ para el offset dentro de la página. Esto nos permite concluir que las páginas son de $2^{13} \text{ bytes} = 8192 \text{ bytes}$.

- Referencia: 122881 => Página: $122881 / 8192 = 15$
- Referencia: 98427 => Página: $98427 / 8192 = 12$
- Referencia: 93112 => Página: $93112 / 8192 = 11$
- Referencia: 156191 => Página: $156191 / 8192 = 19$

Pag =>	15	12	11	19	
28 M	15 UM	15 UM	15 UM	15 UM	
16 M	16 M <=	16 M <=	11 UM	11 UM	
9 UM <=	9 M	9 M	9 M <=	19 UM	
12 U	12	12 M U	12 M U	12 M U <=	
-	- Page fault - Escritura página 28 2 accesos a disco	-	- Page fault - Escritura página 16 2 accesos a disco	- Page fault - Escritura página 9 2 accesos a disco	Total: 6 accesos a disco.

- 4- a) Se solicita completar el nuevo inodo y representar el contenido de los últimos 2 bloques. Mencionar la cantidad de bloques de datos que puede alcanzar un bloque de punteros.

Observando el contenido de los bloques y el contenido del puntero directo del inodo, se desprende que los tamaños de cada uno son:

- **Tamaño del puntero:** 4 Bytes
- **Tamaño del cluster / bloque:** 16 Bytes
- **Cantidad de bloques de datos que puede alcanzar un bloque de punteros:** 4

Considerando que el archivo en fat comienza en el bloque 1h, leyendo la tabla se puede concluir que los clusters que lo comprenden son:

FAT: 00000001h -> 00000002h -> 00000003h -> 00000005h

Al copiar los 4 bloques a EXT, es necesario utilizar un bloque extra de punteros, dada la conformación del inodo en este sistema, por lo tanto, se asignarán los bloques:

EXT2: 00000065h -> 00000066h -> 00000067h -> **00000068h** -> 00000069h

Punteros del Inodo

PD	00000065h
PD	00000066h
PD	00000067h
IS	00000068h
ID	-

Bloque 00000068h

00000005h	-
-	-

Bloque 00000069h

BI	EN	VE	NI
DO	S-	A-	SO

b) Si el pendrive almacena 2GB, justificar el tamaño de la Tabla FAT y el teórico del FS.

Tamaño máximo teórico del FS: $2^{28} * 2^4 = 2^{32} \rightarrow 4\text{GB}$

Tamaño de la tabla FAT: $\rightarrow 2^{31} / 2^4 \rightarrow 2^{27}$ entradas $\rightarrow 2^{27} * 2^2 \rightarrow 512\text{ MB}$

c) Dado que el archivo es copiado desde un filesystem donde no existen permisos para los archivos, hacia uno donde sí existen, es necesario asignarle permisos al nuevo archivo, cuyos valores podrían ser especificados por el usuario que los copia, o definidos por defecto según la configuración del sistema.

d) Estructuras administrativas accedidas:

- FAT32: tabla fat
- EXT2: bitmap de inodos, tabla de inodos, bitmap de bloques, superbloque, descriptores de grupo